
Supervised and Self-Supervised Learning on Food-251

Anonymous Author(s)

Affiliation, Address

anon.email@example.org

Abstract

1 We train a residual convolutional network from scratch to classify the 251 fine-
2 grained food categories of FoodX-251, under a hard budget of ten million trainable
3 parameters and a single 8GB laptop GPU. We search among parameter-matched
4 trunk variants and pooling heads using short, fixed-seed proxy runs on a held-
5 out development split, then tune a training recipe – learning rate, augmentation,
6 Mixup/CutMix, an exponential moving average, a noise-robust loss, batch size, and
7 crop scale – with the same proxy protocol before confirming the result with one
8 full-length run. The tuned recipe reaches **63.83% top-1 / 81.79% top-3 / 87.39%**
9 **top-5** on a held-out test split touched exactly once, up from 61.57% top-1 for the
10 same trunk under an untuned recipe. We outline a planned comparison against self-
11 supervised pretraining under an equal-GPU-hour budget, reported once measured.
12 Throughout, we find that measured wall-clock and memory behavior on commod-
13 ity hardware – channel-width alignment, spatial resolution, and thermal throttling
14 – shape the final design as much as raw parameter or FLOP counts do, and that
15 most proposed recipe improvements do not survive a fixed-seed proxy and a paired
16 significance test at this parameter budget.

17 1 Introduction

18 FoodX-251 is a fine-grained food image classification benchmark spanning 251 categories, built from
19 a large, web-crawled, noisily labeled training set with a smaller, manually curated and clean valida-
20 tion set [1]. Fine-grained food categories are hard to separate because inter-class visual differences
21 (plating, preparation, garnish) can be subtler than the intra-class variation induced by lighting, angle,
22 and portioning, and the label noise inherent to a web-crawled training set compounds the problem.

23 This work targets FoodX-251 under two constraints imposed by the setting rather than chosen for
24 effect: the classifier is trained from scratch, with no ImageNet or other pretraining, under a budget
25 of fewer than 10,000,000 trainable parameters, and every run happens on a single consumer laptop
26 GPU with 8GB of VRAM. Neither constraint is negotiable, and together they shape everything
27 that follows: an architecture search that screens on measured throughput and memory before ever
28 measuring accuracy (Section 5.1), and a training recipe built to spend the wall-clock the 8GB budget
29 buys, not just the parameter budget itself.

30 We make four contributions, in the order the underlying work was carried out: a measured, budget-
31 constrained architecture search over residual-trunk variants and pooling heads, including a check of
32 the parameter budget below the adopted trunk as well as above it (Section 5.1); a from-scratch training
33 recipe modernized with mixed-precision training, a cosine schedule, and label smoothing, then tuned
34 across learning rate, augmentation, Mixup/CutMix, an exponential moving average, a noise-robust
35 loss, batch size, and crop scale, each checked against the untuned recipe with a paired significance
36 test rather than a point estimate alone (Section 5.2); a full-length training run confirming the tuned
37 recipe on a test split read exactly once (Section 5.3); and a planned comparison of supervised training
38 against self-supervised pretraining under an equal-GPU-hour budget (Section 5.4).

39 One design choice is worth flagging before the rest. Despite FoodX-251’s long-tailed reputation, the
40 training split we measured is close to class-balanced (median 471 images per class; a single 34-
41 image class drives the oft-cited 19.3x imbalance ratio), so class-balanced losses, logit adjustment,
42 and long-tail-specific training are out of scope here – the real distributional problem is label noise,
43 not long-tailedness (Section 3). More generally: the architecture search, the recipe tuning, and one
44 full training run confirming the tuned recipe are complete at the time of writing; the self-supervised
45 comparison is in progress, and is marked accordingly below rather than reported with numbers we
46 have not measured.

47 2 Related work

48 We organize related work into three lanes, matching the three axes of this project: architecture,
49 training recipe, and pretraining objective.

50 **Parameter-efficient architecture design.** Recent efficient-CNN work targets mobile and edge
51 deployment under similar per-parameter and per-FLOP pressure: MobileNetV4 [2] and MobileOne
52 [3] optimize explicitly for on-device latency rather than parameter count alone, and ConvNeXt V2
53 [4] revisits pure-convolutional design at a scale where masked-autoencoder pretraining is available
54 – a regime this project’s from-scratch constraint excludes. None of these target fine-grained food
55 classification, and our own literature search turned up no source addressing FoodX-251, Food-101,
56 or Food-2K under a hard parameter budget (research/2026-07-30-architecture-brief.md) – the
57 architecture decisions in Section 4 rest on general efficient-CNN principles and our own measure-
58 ments rather than a food-specific prior.

59 **Augmentation and label-noise robustness.** RandAugment [5] and TrivialAugment [6] establish
60 that automated or tuning-free augmentation policies match or beat searched policies at near-zero
61 search cost; mixup [7] and CutMix [8] are complementary regularizers that mix inputs and labels
62 rather than choosing among fixed operators. Generalized Cross Entropy [9] gives a noise-robust
63 alternative to label-smoothed cross-entropy, directly relevant to FoodX-251’s web-crawled training
64 labels. As with the architecture literature, none of these ablate sub-10M-parameter models trained
65 for tens of epochs (research/2026-07-30-augmentation-noise-brief.md) – whether the accepted
66 wisdom transfers to that regime is precisely what Section 5.2 is designed to check.

67 **Self-supervised pretraining.** SimCLR [10] established contrastive pretraining as a strong alter-
68 native to supervised training but relies on a large negative-sample batch to work well; at this project’s
69 8GB budget, reachable batch sizes fall below where SimCLR is known to degrade, which is why
70 it is excluded here. BYOL [11], SimSiam [12], and Barlow Twins [13] instead avoid negative pairs
71 entirely and remain viable at small batch sizes, making them the basis for the pretraining design in
72 Section 5.4.

73 3 Dataset and protocol

74 FoodX-251 [1] contains 251 fine-grained food categories: a large, web-crawled training set with noisy
75 labels, and a smaller, manually curated validation set of 11,994 images.

76 **Class balance.** Despite FoodX-251’s long-tailed reputation, per-class training counts are close to
77 balanced: median 471 images per class, 10th/90th percentile 366/580, and exactly one class below
78 200 images (34) – that single outlier drives the frequently cited 19.3x imbalance ratio. We therefore
79 do not use class-balanced losses, logit adjustment, or decoupled classifier retraining; the real distrib-
80 utional challenge is label noise, since training labels are web-crawled and validation labels are clean,
81 not long-tailedness.

82 **Validation split.** Ablations and checkpoint selection must never touch the number this report
83 ultimately quotes, or that number is implicitly selected-on. We split the 11,994-image validation set
84 50/50, stratified per class (every one of the 251 classes lands within one image of an exact half), into
85 val-dev (6,063 images) and val-test (5,931 images), deterministically: src/make_val_split.py
86 is a pure function of the official validation labels and a fixed seed (251), so the split is byte-identical
87 on every regeneration and is not committed (splits/val_split.csv is gitignored and regenerated

88 once per checkout). Every ablation and architecture comparison in this report reads val-dev; val-
89 test is read exactly once, for the headline number in Section 5.3.

90 **Proxy protocol.** At the measured throughput of the selected trunk (about 1.2 minutes/epoch,
91 Section 5.1), a 15-epoch run costs about 18 minutes, making comparisons that would be unaffordable
92 at full length routine. Every architecture and recipe comparison below ranks candidates on a fixed-
93 seed, 15-epoch val-dev proxy; the winning configuration is then confirmed with one full-length run.
94 A point-estimate gap between two 15-epoch proxies does not by itself say whether the comparison
95 had the power to detect a real difference, so most comparisons below additionally report a paired
96 McNemar’s test between the two checkpoints’ per-image correctness on the shared val-dev images
97 (benchmarks/significance_test.py) – discordant predictions only, two-sided exact binomial on
98 the discordant pairs.

99 **Seeding.** main.py accepts a --seed argument, but no run behind any number in this report passed
100 one: every architecture and recipe comparison is a single, unseeded run per configuration. This was
101 found only after most of the recipe search had already run (tracked as a known gap rather than silently
102 left unaddressed), and it means every accuracy figure below is a point estimate, not an exactly repro-
103 ducible one – a limitation we return to in Section 6 rather than paper over by re-running everything
104 under a fixed seed after the fact.

105 **Logging.** Every run cited in this report is logged with a config hash, per-epoch train/validation
106 top-1 and top-5, wall-clock time, and peak VRAM (src/runlog.py), so every number below traces
107 back to a specific run.

108 4 Method

109 This section describes FoodCNN’s architecture, then the training recipe used to fit it.

110 4.1 Architecture

111 FoodCNN (src/model.py) is a plain residual CNN, sized to spend its parameter budget where it
112 can affect accuracy rather than where convention places it. A stem (3x3 stride-2 convolution, batch
113 normalization, ReLU, 3x3 stride-2 max-pool) takes the input to a quarter of its input resolution before
114 any residual stage runs. Four residual stages follow, each halving spatial resolution and doubling
115 channel width relative to the last – 64, 128, 256, then 512 channels, with block counts [2, 2, 2, 1] –
116 so that the final stage, carrying one block rather than two, is what keeps the trunk under budget.

117 Each residual block places batch normalization after both convolutions in the main branch and,
118 whenever the shortcut is not the identity, after the 1x1 shortcut projection as well – an unnormalized
119 branch should not be added to a normalized one. The second batch norm’s scale (γ) in every block is
120 zero-initialized, so every block begins training as an identity map; this is what makes a high learning
121 rate safe under warmup. The nonlinearity is applied after the residual addition, not before it, so each
122 block is an actual nonlinearity rather than an affine detour.

123 **A genuinely global head.** Pooling to (1, 1) rather than the (7, 7) map convolutions naturally leave
124 behind matters: at (7, 7), flattening produces a $512 \times 7 \times 7 = 25,088$ -feature vector, and a single
125 subsequent linear layer to 251 classes costs 6.4M parameters – 94% of the entire model, spent on
126 its least expressive layer. Pooling to (1, 1) first reduces that same layer to Linear(512, 251), 129k
127 parameters.

128 **Parameter budget.** With the fixes above, FoodCNN totals 6,576,955 trainable parameters – 65.8%
129 of the 10M budget – with roughly 98% of them inside the convolutional trunk rather than the
130 classifier head.

131 **A rejected alternative.** A depthwise-separable version of the same trunk (MobileNetV2-style
132 inverted residuals, retained in src/model.py as a documented negative result rather than deleted) was
133 measured head-to-head with the plain-convolution trunk on this hardware: 2.55M parameters, 621
134 img/s, 3.83 GiB peak memory, against the plain-convolution trunk’s 6.58M parameters, 918 img/s,
135 1.28 GiB peak (full detail in Section C). Depthwise convolutions here cost activation memory and

Table 1: Sub-baseline trunk widths and alternative pooling heads, 15-epoch val-dev proxies under the tuned Phase D recipe (Section 5.2), each compared against the baseline trunk with a paired McNemar’s test on the same val-dev images.

Variant	Params	val-dev top-1	Δ vs. baseline	McNemar p
narrow [2, 2, 2, 1] 64-384	5.18M	55.22%	+0.00	1.0000
baseline [2, 2, 2, 1] 64-512	6.58M	55.22%	–	–
head: spatial attention	6.59M	55.01%	–0.21	0.6826
narrow [2, 2, 2, 1] 32-256	1.68M	48.64%	–6.58	< 0.0001
head: GAP+GMP concat	6.71M	37.51%	–17.71	< 0.0001

wall-clock far more than they save in parameter count, and under this project’s constraints, throughput and memory – not parameter count – are what actually bind (Section 6).

The parameter budget is not what binds, on either side of the adopted trunk. Section 5.1 only ever swept trunks from 6.58M parameters up; a later pass (Table 1) checked below it and tested the pooling head itself for the first time. A narrower trunk ([2, 2, 2, 1] 64-384, 5.18M parameters, 21% fewer than baseline) ties the baseline exactly under a paired significance test ($p = 1.0000$), and only a further 3.5M-parameter cut (32-256, 1.68M parameters) costs accuracy, and does so sharply (–6.58 points, $p < 0.0001$). Neither a spatial-attention pooling head nor a concatenated GAP+GMP head improves on plain global average pooling; the concatenated head’s apparent underperformance is attributable to an untuned learning rate for its differently scaled inputs rather than a capacity problem (Section 6). Together with Section 5.1’s finding that 9.46M parameters buys at most 1.17 points over the baseline on a single seed, this places baseline on a plateau extending from at least 5.18M to 9.46M parameters – the parameter cap is not the binding constraint at this trunk depth in either direction, which is why Section 5.2, not further architecture search, is where the remaining project budget goes.

4.2 Training recipe

The reference training loop is modernized in ways that cost nothing at this parameter budget: `torch.autocast("cuda", dtype=torch.bfloat16)` without a `GradScaler` (bf16 does not need dynamic loss scaling), `channels_last` memory format on both model and inputs, a cosine learning-rate schedule with five-epoch linear warmup in place of step decay, `CrossEntropyLoss` with label smoothing 0.1, SGD with Nesterov momentum, eight dataloader workers (up from four), a 176px training crop (down from 224), and no `DataParallel` wrapper – there is only one GPU to shard across, and the wrapper’s scatter/gather adds a per-step cost and a `module.prefix` to every checkpoint key for no benefit here. Two hyperparameters this loop leaves open – peak learning rate and the crop’s minimum scale – are swept rather than guessed; Section 5.2 reports the search and the values it settled on: batch 256, learning rate 0.8, and the default `RandomResizedCrop` scale floor of 0.08 kept unchanged despite starting from an ImageNet-scale assumption. Validation reads a 224px center crop from a 256px resize regardless of training resolution, a FixRes-style correction [14] for the systematic scale shift `RandomResizedCrop` introduces between train and test time; because it is applied by default rather than as an opt-in flag, it is already folded into every accuracy figure in this report rather than a separate, uncoded improvement.

4.3 Recipe extensions

Four further axes are implemented as opt-in flags to the training script, all defaulting to off so that nothing about the recipe above changes unless explicitly requested: an augmentation switch between no augmentation, `TrivialAugment` [6], and `RandAugment` [5]; batch-wise Mixup [7] and CutMix [8], scored against the pre-mix hard label since a mixed batch has no single correct class to evaluate top-1/top-5 against; an exponential moving average of model weights (`torch.optim.swa_utils.AveragedModel`, batch-norm running statistics left live rather than averaged); and Generalized Cross Entropy [9] as an alternative to label-smoothed cross-entropy for the training set’s label noise. Section 5.2 reports which, if any, improve on the recipe above.

184 Table 2: 15-epoch val-dev proxy results for four candidate trunks, all under the 10M-parameter cap.

185	Variant	Params	val-dev top-1	val-dev top-5	Peak VRAM
186	[2,2,2,2] 64-448	9.46M	49.99%	78.29%	2.04 GiB
187	[2,2,4,1] 64-512	8.94M	49.73%	77.93%	2.14 GiB
188	baseline [2,2,2,1] 64-512	6.58M	48.82%	77.12%	1.99 GiB
189	5-stage 48-512 [2,2,2,2,1]	9.08M	47.42%	75.94%	1.61 GiB

190 5 Experiments

191 Accuracy figures below are val-dev top-1/top-5 unless stated otherwise; val-test is read exactly
 192 once, in Section 5.3, per the protocol in Section 3. All measurements were taken on the project GPU
 193 (RTX 5050 Laptop, 8GB VRAM) in its performance power profile; Section 6 explains why that
 194 qualifier matters for every throughput number below.

195 5.1 Architecture search

196 A first pass screened trunk variants by synthetic throughput and memory alone (benchmarks/
 197 bench.py, benchmarks/trunk_variants.py, full sweep in Section B), before spending any GPU-
 198 hours on accuracy. Two findings from that pass decided which variants were worth an accuracy proxy
 199 at all. First, stage widths must be multiples of 32: a 72-144-288-576 trunk has **fewer** parameters than
 200 the five-stage candidate below yet runs 46% slower, falling off the tensor-core fast path. Second, final
 201 spatial resolution dominates cost more than any other single knob: dropping the stem’s max-pool to
 202 preserve a 6x6 final feature map, at identical parameter count, costs 2.9x throughput (1960 to 684
 203 img/s) and doubles peak memory.

204 Four candidates under the parameter cap were then measured for accuracy with fixed-seed, 15-epoch
 205 val-dev proxies (benchmarks/proxy_sweep.py):

206 The five-stage variant’s extra downsample to a 3x3 final map costs 1.4-2.6 accuracy points despite
 207 being the second-largest candidate by parameter count, so the throughput and memory advantage
 208 it showed in the synthetic pass does not transfer to a real training signal; it is ruled out. The
 209 remaining three sit within 1.2 points of each other on a single seed each – not enough to call
 210 a winner with confidence. We adopt the baseline [2,2,2,1] 64-512 trunk provisionally: it is
 211 statistically indistinguishable from the other two here, uses 30% fewer parameters than the nominal
 212 leader, and that headroom has not measured as worth anything yet – better spent, for now, on the
 213 recipe tuning in Section 5.2. [2,2,4,1] and [2,2,2,2] 64-448 remain defined in benchmarks/
 214 trunk_variants.VARIANTS and can be re-proxied without repeating the measurement work above,
 215 if the tuned baseline underperforms expectations.

216 5.2 Recipe ablation

217 Six recipe axes were each swept independently against the same 15-epoch val-dev control, all on
 218 the baseline trunk. Every gap below is reported with the McNemar significance test from Section 3;
 219 several axes with a real-looking point-estimate gap did not survive it.

220 **Learning rate.** Doubling the learning rate five times over {0.05, 0.1, 0.2, 0.4, 0.6, 0.8} produced
 221 a clean diminishing-returns curve (+5.45, +3.15, +1.46, +0.94, +0.62 top-1 points per doubling),
 222 so the search stopped at 0.8 rather than continue into a regime of shrinking reward and growing
 223 optimization noise (0.8’s val-accuracy curve is visibly noisier mid-run than 0.4’s or 0.6’s, despite a
 224 smooth, undiverged training loss):

225 **Augmentation.** TrivialAugment and RandAugment each score roughly 3 points below a no-
 226 augmentation control at 15 epochs, and a deep-dive (macro-F1, weighted-F1, worst-30-class average,
 227 calibration) confirms this is not a top-1-only illusion – the control leads on every axis, so augmen-
 228 tation is not quietly buying rare-class recall at the cost of overall accuracy. But both augmented runs’

Table 3: Learning-rate sweep, 15-epoch val-dev proxy, batch 256.

LR	val-dev top-1	top-3	top-5
0.05	43.96%	–	–
0.10 (prior default)	49.41%	–	–
0.20	52.56%	–	–
0.40	54.02%	74.16%	81.02%
0.60	54.96%	–	–
0.80 (adopted)	55.58%	–	–

Table 4: RandAugment vs. no augmentation, matched at 30 epochs, same protocol.

Recipe (30 ep)	val-dev top-1	macro-F1	weighted-F1	worst-30 avg	ECE
none (control)	61.27%	59.38%	60.90%	23.62%	0.144
RandAugment	60.43%	58.31%	59.84%	23.21%	0.157

Table 5: Mixup/CutMix/EMA/GCE, 15-epoch val-dev proxy against the same control as Table 3.

Recipe	val-dev top-1	macro-F1	worst-30 avg	zero-acc classes	ECE
none (control)	55.58%	53.38%	16.77%	4	0.182
Mixup [7]	42.95%	40.05%	5.13%	11	0.231
CutMix [8]	48.84%	45.61%	7.19%	5	0.172
EMA (decay 0.999)	39.88%	38.32%	4.52%	9	0.107
GCE [9] ($q = 0.7$)	22.23%	14.86%	0.00%	119	0.328

training loss was still well above the control’s at epoch 15, so a matched-epoch rematch at 30 epochs was run before drawing a conclusion:

The 15-epoch proxy exaggerated the gap (about 3 points vs. 0.84 at matched epochs), and the control leads on every point estimate in the rematch – but a paired McNemar’s test on the 855 discordant val-dev predictions puts the 0.84-point gap at $p = 0.075$, not significant. The honest read is that this proxy cannot distinguish “control is slightly better” from “the two are tied”; the plain recipe carries forward on simplicity and an unbeaten record, not a proven margin over RandAugment specifically.

Mixup, CutMix, EMA, and Generalized Cross Entropy. Unlike augmentation, these four are decisively ruled out – all four land at $p < 0.0001$ against the same 15-epoch control:

GCE’s collapse is not underconfidence hiding behind a low top-1: mean confidence on its wrong predictions is 0.49 and its ECE is the worst of every recipe tested, i.e. it is confidently wrong on the 119 of 251 classes it collapsed on. Both GCE and EMA’s failures are attributable to hyperparameters tuned for a different loss landscape – learning rate 0.8 was found for label-smoothed cross-entropy, and GCE’s bounded loss produces much smaller gradients as confidence rises, while EMA’s 0.999 decay implies a roughly 1,000-step averaging window against a 6,945-step proxy – rather than a settled verdict against either technique; this is flagged as an open question in Section 6, not reopened here.

Batch size. Throughput is flat from batch 160 to 768 (1738-1785 img/s), so this axis is a gradient-noise and batch-norm-statistics question rather than a throughput one. Swept at $\{160, 256, 512\}$ with learning rate scaled linearly from the batch-256 control, all three are statistically tied ($p = 0.74$ and

Table 6: Tuned recipe, full 90-epoch run. val-test is read once, here; every other number in this report reads val-dev.

	top-1	top-3	top-5
val-dev, best (epoch 86)	64.36%	82.22%	87.51%
val-dev, final (epoch 90)	64.26%	82.14%	87.61%
val-test (headline)	63.83%	81.79%	87.39%

$p = 0.49$ respectively against the control); batch 256 is kept because every other axis in this section was measured there, on no evidence against it.

Crop scale floor. RandomResizedCrop’s minimum crop area had run at the ImageNet-inherited default of 0.08 through every result above – aggressive for this dataset’s roughly 90k training images. Raising the floor to 0.25 and 0.40 was motivated by a diagnosis that recurs throughout this section: validation accuracy exceeds training accuracy in every recipe tried, the opposite of the usual overfitting pattern, and the gap widens rather than closes with more epochs. Raising the crop floor does close that gap (val-minus-train goes from +11.05 points at 0.08 to +4.31 at 0.25 to −0.56 at 0.40, ordinary train-above-val behavior by the third), but accuracy does not follow: 0.25 ties the control and 0.40 is significantly **worse** ($p = 0.0232$). RandomResizedCrop’s low-scale cropping is doing double duty as both a regularizer and a view-diversity multiplier that nothing else in the recipe replaces, so relaxing it trades away diversity without buying back the epochs needed to use the easier task – a net loss at a fixed 15-epoch budget. The default of 0.08 is kept.

A further candidate, not part of the six axes above. A similarity-smoothed cross-entropy loss – label smoothing redistributed toward classes flagged as likely near-duplicates by bidirectional confusion in the trunk’s own error analysis (Section 6), rather than spread uniformly – was implemented and logged at the same protocol (lr 0.8, batch 256, 15 epochs): 56.06% val-dev top-1, the best 15-epoch number measured anywhere in this search. It has not been checked against the control with a significance test or replicated, so it is reported here as a logged, promising result rather than an adopted part of the recipe – the standard this section holds every other axis to, and a natural next step before the self-supervised comparison in Section 5.4.

Adopted recipe. Label-smoothed cross-entropy, no extra augmentation, no Mixup/CutMix, no EMA, learning rate 0.8, batch 256, RandomResizedCrop scale floor 0.08 – unmodified from the base recipe in Section 4 except for the tuned learning rate. This is the recipe Section 5.3’s headline run trains with.

5.3 Full supervised training run

Pipeline validation, pre-tuning. Before spending GPU-hours tuning the recipe, a full 90-epoch run on the unmodified baseline trunk with the base recipe (learning rate 0.1, none of Section 5.2’s tuning) confirmed the pipeline survives full length. It converged to **61.57% val-dev top-1 / 85.67% top-5** (best checkpoint at epoch 82: 62.08% / 85.52%), rising from about 7% at epoch 0 and comfortably past both a from-scratch Food-101 anchor (mid-50s%) and this trunk’s own 15-epoch proxy (Table 2, 47-50%). No crashes, NaNs, or out-of-memory errors occurred; peak memory was 1.99 GiB, still far under the 8 GiB ceiling at full length. Wall-clock was 2.31 hours, 28% above the approximately 1.8 hour synthetic estimate from the architecture search’s throughput sweep – per-batch time rose from about 0.16s early in the run to about 0.26s past epoch 60 as sustained load heat-soaked the laptop GPU (Section 6).

Tuned headline run. With Section 5.2’s adopted recipe – unchanged trunk and head, learning rate 0.8, batch 256, crop-scale floor 0.08 – a second full 90-epoch run was trained and read against val-test exactly once, per Section 3:

This is up 2.3 points on the untuned pipeline-validation run above, at the same trunk and epoch count, and continues that run’s trend cleanly across 15/30/90-epoch schedules (55.6% to 61.3% to 63.8% val-test / 64.4% val-dev best) – diminishing but still real returns from schedule length alone, consistent with the cosine schedule’s own design. It also confirms the underfitting diagnosis

Section 5.2’s crop-scale investigation was built on: the +11.05-point val-over-train gap measured at epoch 15 has closed to near zero by epoch 90 (train top-1 64.88% vs. val-dev top-1 64.26% at the final epoch) – the model needed the full schedule to converge, and nothing in Section 5.2 sped that up because, per the crop-scale result, nothing needed to. Best and final checkpoints are within 0.1 points of each other here, unlike the roughly 0.5-point gap in the pipeline-validation run, itself evidence against late-training overfitting. Wall-clock was 2h05m at 1.99 GiB peak memory, with no thermal slowdown this time (0.147-0.169s/step steady from epoch 5 through epoch 89) after the machine was switched to its high-performance power profile partway through an earlier proxy run this phase – consistent with, though not a controlled replication of, the pipeline-validation run’s own read that its slowdown was a power-cap effect rather than a hard thermal limit.

As in every result in this report (Section 3), this run was not seeded; the headline number above is a point estimate.

5.4 Self-supervised comparison

The design is fixed and implemented; no pretraining run has completed at the time of writing. SimSiam [12] is chosen over BYOL [11]: BYOL’s target network is a second, momentum-averaged copy of the trunk, the same mechanism implicated in Table 5’s EMA collapse, and SimSiam avoids it entirely with a stop-gradient and predictor asymmetry instead, at no second 6.5M-parameter shadow copy in an already-tight 8GB budget. SimCLR [10] is excluded outright: it is known to degrade below roughly 1,000-example batches, unreachable here.

Two design choices depart from a naive port of the supervised pipeline. First, pretraining draws on the FoodX-251 challenge’s 28,377-image unlabeled test split in addition to the 118,475 labeled training images (146,852 total) – pretraining on only the images already carrying labels would not exercise the premise a self-supervised comparison exists to test. Second, because SimSiam’s training loss can and does keep falling even after the representation has collapsed to a constant output, pretraining is monitored with a weighted k -nearest-neighbor probe on frozen features [15] against val-dev, alongside two collapse diagnostics – mean per-dimension feature standard deviation and the effective rank of the feature covariance – computed every few epochs rather than trusting loss alone.

The plan calls for 200 pretrain epochs at 176px (about 8 GPU-hours), finetuning at 176px (about 1.8 GPU-hours), against an equal-GPU-hour supervised control (about 8 GPU-hours) on the same trunk – without that control, the comparison cannot separate “self-supervised pretraining helped” from “this trunk keeps improving with more epochs regardless of objective.” Results, and the finetuning protocol they are read under, replace this paragraph once measured.

6 Discussion

Hardware measurement mattered as much as the architecture. Several findings above only exist because throughput and memory were measured rather than assumed. Stage widths must be multiples of 32 to stay on the tensor-core fast path (a 46% throughput gap at equal parameter count, Section 5.1); final spatial resolution costs more than almost any other knob (2.9x throughput, 2x memory, for an unchanged parameter count); the dataloader is never the bottleneck at this resolution – plain crop-and-flip sustains 4.4-4.7k images/second at any worker count against roughly 1.7-1.9k images/second of GPU demand, and even TrivialAugment keeps 2.5x headroom at eight workers, falling to 1.4x at sixteen as oversubscription costs throughput on this machine’s 15GB of system RAM; and batch size is throughput-flat from 160 to 768 (1738-1785 img/s) while memory scales linearly from 1.28 to 5.64 GiB, meaning the GPU is compute-saturated at the smallest batch size tried, not idle.

Thermal throttling makes a short benchmark an upper bound, not an estimate. The GPU runs against a roughly 50W power cap with its clock oscillating 2175-2340 MHz against a 3090 MHz maximum, and an early version of the throughput benchmark swung 45% on one variant between sweeps before being changed to time three 40-step rounds and keep the best (sweeps now agree within 2-5%, and rankings are stable). Even so, the full run in Section 5.3 took 28% longer than its own synthetic estimate: sustained 90-epoch load heat-soaks this laptop GPU in a way no short benchmark window reproduces. Every throughput figure in this report is an upper bound on real epoch time and a ratio between variants, not a wall-clock promise.

Near-duplicate classes are a real dataset property, not a training artifact. Bidirectional confusion analysis on the recipe-search checkpoints repeatedly flags the same class pairs as mutually confused – beef tartare/steak tartare, chicken wing/buffalo wing, oyster/huitre, among others – and these pairs recur under all three augmentation conditions in Table 4, not just one run. That consistency across independently trained checkpoints is what makes them a property of the label space rather than a fluke of one training run, and is the motivation behind Section 5.2’s similarity-smoothed loss candidate.

Ruled-out recipe axes may only be ruled out at this learning rate. GCE and EMA’s collapses in Table 5 are large and statistically decisive, but both use a learning rate of 0.8, tuned for label-smoothed cross-entropy’s loss landscape rather than either technique’s own. A bounded loss like GCE produces systematically smaller gradients as confidence rises, and EMA’s effective averaging window is a function of its decay rate relative to schedule length; either could plausibly change under its own learning-rate search. This report treats them as ruled out under the recipe actually adopted, not as a claim that no learning rate would make them competitive.

Limitations. The architecture comparison in Section 5.1 rests on a single 15-epoch seed per variant; the 1.2-point spread separating three of the four candidates is well within plausible run-to-run noise, which is why the baseline trunk is adopted provisionally rather than declared a winner outright. More broadly, no run in this report – across architecture search, recipe ablation, or the full 90-epoch confirmation – was seeded (Section 3), so every accuracy figure here is a point estimate from a single training run rather than a distribution; where a McNemar test is reported it addresses sampling variance on the shared validation images, not this run-to-run variance. All measurements are specific to one machine (RTX 5050 Laptop, 8GB VRAM) and need not transfer, even in ratio, to other hardware. Training is from scratch throughout, by the project’s own constraint, so absolute accuracy figures here are not directly comparable to pretrained food-classification baselines without accounting for that.

7 Conclusion

Under a 10M-parameter budget and a single 8GB laptop GPU, a plain residual CNN with a genuinely global pooling head reaches **63.83% val-test top-1 / 81.79% top-3 / 87.39% top-5** on FoodX-251 – up from 61.57% val-dev top-1 for the same trunk under an untuned recipe – after a six-axis recipe search in which most proposed improvements (stronger augmentation, Mixup, CutMix, an exponential moving average, a noise-robust loss, a wider crop-scale floor) failed a paired significance test against the plain baseline, and a parameter-budget check in both directions found the trunk sitting on a plateau rather than starved of capacity. Reaching the headline number meant treating measured wall-clock and memory behavior – channel-width alignment, spatial resolution, thermal throttling – as design constraints alongside the parameter cap itself: a fifth residual stage and a depthwise-separable trunk both looked attractive on paper and measurably lost once actually run. The self-supervised comparison against an equal-compute supervised control (Section 5.4) remains open.

TODO — Revisit this conclusion once Section 5.4 has real results.

References

- [1] P. Kaur, K. Sikka, W. Wang, S. Belongie, and A. Divakaran, “FoodX-251: A Dataset for Fine-grained Food Classification,” *arXiv preprint arXiv:1907.06167*, 2019, [Online]. Available: <https://arxiv.org/abs/1907.06167>
- [2] D. Qin *et al.*, “MobileNetV4 - Universal Models for the Mobile Ecosystem,” *arXiv preprint arXiv:2404.10518*, 2024, [Online]. Available: <https://arxiv.org/abs/2404.10518>
- [3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, “MobileOne: An Improved One millisecond Mobile Backbone,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7907–7917. [Online]. Available: <https://arxiv.org/abs/2206.04040>
- [4] S. Woo *et al.*, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16133–16142. [Online]. Available: <https://arxiv.org/abs/2301.00808>

- [5] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020, pp. 702–703. [Online]. Available: <https://arxiv.org/abs/1909.13719>
- [6] S. G. Müller and F. Hutter, “TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.10158>
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.09412>
- [8] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.04899>
- [9] Z. Zhang and M. Sabuncu, “Generalized Cross Entropy Loss for Training Deep Neural Networks with Noisy Labels,” in *Advances in Neural Information Processing Systems*, 2018. [Online]. Available: <https://arxiv.org/abs/1805.07836>
- [10] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 1597–1607. [Online]. Available: <https://arxiv.org/abs/2002.05709>
- [11] J.-B. Grill *et al.*, “Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning,” in *Advances in Neural Information Processing Systems*, 2020, pp. 21271–21284. [Online]. Available: <https://arxiv.org/abs/2006.07733>
- [12] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.10566>
- [13] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, “Barlow Twins: Self-Supervised Learning via Redundancy Reduction,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 12310–12320. [Online]. Available: <https://arxiv.org/abs/2103.03230>
- [14] H. Touvron, A. Vedaldi, M. Douze, and H. Jégou, “Fixing the train-test resolution discrepancy,” *Advances in Neural Information Processing Systems*, vol. 32, 2019, [Online]. Available: <https://arxiv.org/abs/1906.06423>
- [15] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, “Unsupervised Feature Learning via Non-Parametric Instance Discrimination,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. [Online]. Available: <https://arxiv.org/abs/1805.01978>

A Technical appendices and supplementary material

Supplementary tables referenced from Section 4 and Section 5.1 follow: the full synthetic throughput sweep behind the architecture search, and additional detail on the rejected depthwise-separable trunk.

B Full architecture-screening sweep

Table 2 in the main text reports 15-epoch accuracy proxies for four trunk variants. Those four were selected from a larger synthetic screening pass over throughput, parameter count, and memory alone (benchmarks/bench.py, benchmarks/trunk_variants.py), all at 176px training resolution and batch 160, before any accuracy was measured. The full sweep:

The last row isolates the resolution effect cited in Section 6: identical parameter count to the row above it, but with the stem’s max-pool removed to keep the final feature map at 6x6 instead of 3x3, at a cost of 2.9x throughput and roughly 2x peak memory. The “uniformly wider 72-576” row isolates the tensor-core alignment effect: fewer parameters than the five-stage variant, at 46% lower throughput, purely from using channel widths that are not multiples of 32.

C The rejected depthwise-separable trunk

467 Table 7: Synthetic throughput/memory sweep over trunk variants under the 10M-parameter cap,
 468 176px training resolution, batch 160. “90 ep” is the projected wall-clock for a full 90-epoch run at
 469 the measured throughput; “Final map” is the trunk’s spatial resolution immediately before global
 470 average pooling.

471	Variant	Params	img/s	90 ep	Peak VRAM	Final map
472	baseline [2,2,2,1] 64-512	6.58M	1688	1.8 h	1.26 GiB	6x6
473	[2,2,3,1] 64-512	7.76M	1585	1.9 h	1.31 GiB	6x6
474	[2,2,4,1] 64-512	8.94M	1475	2.0 h	1.36 GiB	6x6
475	[2,3,3,1] 64-512	8.06M	1460	2.0 h	1.39 GiB	6x6
476	[2,2,2,2] 64-448	9.46M	1608	1.8 h	1.30 GiB	6x6
477	wider@11 64-320-512	7.98M	1494	2.0 h	1.30 GiB	6x6
478	5 stages 48-512 [2,2,2,2,1]	9.08M	1960	1.5 h	1.04 GiB	3x3
479	uniformly wider 72-576	8.31M	1062	2.8 h	1.41 GiB	6x6
480	5 stages, no stem max-pool	9.08M	684	4.3 h	2.16 GiB	6x6

481 Table 8: Depthwise-separable vs. plain-convolution trunk, measured head-to-head on the project
 482 GPU (176px, batch 160, bf16, channels_last).

483	Trunk	Params	img/s	Peak VRAM
484	Depthwise-separable (MobileNetV2-style)	2.55M	621	3.83 GiB
485	Plain-convolution residual (adopted)	6.58M	918	1.28 GiB

486 Before the residual trunk in Section 4, a MobileNetV2-style depthwise-separable version of the
 487 same network was implemented and measured head-to-head on the same hardware. It is kept in `src/
 488 model.py`, unused by `FoodCNN.features`, as a documented negative result rather than deleted:

489 Fewer parameters did not mean faster or lighter here: depthwise convolutions trade parameter count
 490 for activation memory and wall-clock, and under this project’s binding constraints – throughput and
 491 memory, not raw parameter count – that trade loses.¹ The plain-convolution trunk was adopted on
 492 this basis before the architecture search in Section 5.1 ever began.

¹This comparison’s 918 img/s baseline figure is lower than the 1688 img/s recorded for the same trunk in Table 7. The two were measured in separate benchmarking passes – this comparison predates `benchmarks/bench.py`’s current three-round, keep-the-best methodology – and, per the thermal-throttling behavior discussed in Section 6, absolute throughput on this hardware is not stable across passes. The qualitative conclusion, that the depthwise-separable trunk trades parameters for memory and wall-clock, does not depend on which figure is used.

493 NeurIPS Paper Checklist

494 The checklist is designed to encourage best practices for responsible machine learning research,
495 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
496 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
497 follow the references and follow the (optional) supplemental material. The checklist does NOT count
498 towards the page limit.

499 Please read the checklist guidelines carefully for information on how to answer these questions. For
500 each question in the checklist:

- 501 • You should answer [Yes], [No], or [NA].
- 502 • [NA] means either that the question is Not Applicable for that particular paper or the
503 relevant information is Not Available.
- 504 • Please provide a short (1–2 sentence) justification right after your answer (even for NA).

505 **The checklist answers are an integral part of your paper submission.** They are visible to the
506 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
507 (after eventual revisions) with the final version of your paper, and its final version will be published
508 with the paper.

509 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
510 While “[Yes]” is generally preferable to “[No]”, it is perfectly acceptable to answer “[No]” provided a
511 proper justification is given (e.g., “error bars are not reported because it would be too computationally
512 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
513 “[No]” or “[NA]” is not grounds for rejection. While the questions are phrased in a binary way, we
514 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
515 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
516 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
517 please point to the section(s) where related material for the question can be found.

518 IMPORTANT, please:

- 519 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- 520 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 521 • **Do not modify the questions and only use the provided macros for your answers.**

522 1. Claims

523 Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s
524 contributions and scope?

525 Answer: **[TODO]**

526 Justification: **[TODO]**

527 Guidelines:

- 528 • The answer NA means that the abstract and introduction do not include the claims made in
529 the paper.
- 530 • The abstract and/or introduction should clearly state the claims made, including the contri-
531 butions made in the paper and important assumptions and limitations. A No or NA answer
532 to this question will not be perceived well by the reviewers.
- 533 • The claims made should match theoretical and experimental results, and reflect how much
534 the results can be expected to generalize to other settings.
- 535 • It is fine to include aspirational goals as motivation as long as it is clear that these goals are
536 not attained by the paper.

537 2. Limitations

538 Question: Does the paper discuss the limitations of the work performed by the authors?

539 Answer: **[TODO]**

540 Justification: **[TODO]**

541 Guidelines:

- 542 • The answer NA means that the paper has no limitation while the answer No means that the
- 543 paper has limitations, but those are not discussed in the paper.
- 544 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 545 • The paper should point out any strong assumptions and how robust the results are to
- 546 violations of these assumptions (e.g., independence assumptions, noiseless settings, model
- 547 well-specification, asymptotic approximations only holding locally). The authors should
- 548 reflect on how these assumptions might be violated in practice and what the implications
- 549 would be.
- 550 • The authors should reflect on the scope of the claims made, e.g., if the approach was only
- 551 tested on a few datasets or with a few runs. In general, empirical results often depend on
- 552 implicit assumptions, which should be articulated.
- 553 • The authors should reflect on the factors that influence the performance of the approach. For
- 554 example, a facial recognition algorithm may perform poorly when image resolution is low
- 555 or images are taken in low lighting. Or a speech-to-text system might not be used reliably
- 556 to provide closed captions for online lectures because it fails to handle technical jargon.
- 557 • The authors should discuss the computational efficiency of the proposed algorithms and
- 558 how they scale with dataset size.
- 559 • If applicable, the authors should discuss possible limitations of their approach to address
- 560 problems of privacy and fairness.
- 561 • While the authors might fear that complete honesty about limitations might be used by
- 562 reviewers as grounds for rejection, a worse outcome might be that reviewers discover limita-
- 563 tions that aren’t acknowledged in the paper. The authors should use their best judgment and
- 564 recognize that individual actions in favor of transparency play an important role in devel-
- 565 oping norms that preserve the integrity of the community. Reviewers will be specifically
- 566 instructed to not penalize honesty concerning limitations.

567 **3. Theory Assumptions and Proofs**

568 Question: For each theoretical result, does the paper provide the full set of assumptions and a

569 complete (and correct) proof?

570 Answer: **[TODO]**

571 Justification: **[TODO]**

572 Guidelines:

- 573 • The answer NA means that the paper does not include theoretical results.
- 574 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
- 575 referenced.
- 576 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 577 • The proofs can either appear in the main paper or the supplemental material, but if they
- 578 appear in the supplemental material, the authors are encouraged to provide a short proof
- 579 sketch to provide intuition.
- 580 • Inversely, any informal proof provided in the core of the paper should be complemented by
- 581 formal proofs provided in appendix or supplemental material.
- 582 • Theorems and Lemmas that the proof relies upon should be properly referenced.

583 **4. Experimental Result Reproducibility**

584 Question: Does the paper fully disclose all the information needed to reproduce the main exper-
585 imental results of the paper to the extent that it affects the main claims and/or conclusions of the
586 paper (regardless of whether the code and data are provided or not)?

587 Answer: **[TODO]**

588 Justification: **[TODO]**

589 Guidelines:

- 590 • The answer NA means that the paper does not include experiments.
- 591 • If the paper includes experiments, a No answer to this question will not be perceived well by
592 the reviewers: Making the paper reproducible is important, regardless of whether the code
593 and data are provided or not.
- 594 • If the contribution is a dataset and/or model, the authors should describe the steps taken to
595 make their results reproducible or verifiable.
- 596 • Depending on the contribution, reproducibility can be accomplished in various ways. For
597 example, if the contribution is a novel architecture, describing the architecture fully might
598 suffice, or if the contribution is a specific model and empirical evaluation, it may be
599 necessary to either make it possible for others to replicate the model with the same dataset,
600 or provide access to the model. In general, releasing code and data is often one good way
601 to accomplish this, but reproducibility can also be provided via detailed instructions for
602 how to replicate the results, access to a hosted model (e.g., in the case of a large language
603 model), releasing of a model checkpoint, or other means that are appropriate to the research
604 performed.
- 605 • While NeurIPS does not require releasing code, the conference does require all submissions
606 to provide some reasonable avenue for reproducibility, which may depend on the nature of
607 the contribution. For example
 - 608 (a) If the contribution is primarily a new algorithm, the paper should make it clear how to
609 reproduce that algorithm.
 - 610 (b) If the contribution is primarily a new model architecture, the paper should describe the
611 architecture clearly and fully.
 - 612 (c) If the contribution is a new model (e.g., a large language model), then there should either
613 be a way to access this model for reproducing the results or a way to reproduce the model
614 (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - 615 (d) We recognize that reproducibility may be tricky in some cases, in which case authors
616 are welcome to describe the particular way they provide for reproducibility. In the case
617 of closed-source models, it may be that access to the model is limited in some way (e.g.,
618 to registered users), but it should be possible for other researchers to have some path to
619 reproducing or verifying the results.

620 5. Open Access to Data and Code

621 Question: Does the paper provide open access to the data and code, with sufficient instructions
622 to faithfully reproduce the main experimental results, as described in supplemental material?

623 Answer: **[TODO]**

624 Justification: **[TODO]**

625 Guidelines:

- 626 • The answer NA means that paper does not include experiments requiring code.
- 627 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
628 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 629 • While we encourage the release of code and data, we understand that this might not be
630 possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including
631 code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer “Yes” if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point

722 out that an improvement in the quality of generative models could be used to generate
 723 deepfakes for disinformation. On the other hand, it is not needed to point out that a generic
 724 algorithm for optimizing neural networks could enable people to train models that generate
 725 Deepfakes faster.

- 726 • The authors should consider possible harms that could arise when the technology is being
 727 used as intended and functioning correctly, harms that could arise when the technology is
 728 being used as intended but gives incorrect results, and harms following from (intentional or
 729 unintentional) misuse of the technology.
- 730 • If there are negative societal impacts, the authors could also discuss possible mitigation
 731 strategies (e.g., gated release of models, providing defenses in addition to attacks, mecha-
 732 nisms for monitoring misuse, mechanisms to monitor how a system learns from feedback
 733 over time, improving the efficiency and accessibility of ML).

734 11. Safeguards

735 Question: Does the paper describe safeguards that have been put in place for responsible release
 736 of data or models that have a high risk for misuse (e.g., pre-trained language models, image
 737 generators, or scraped datasets)?

738 Answer: **[TODO]**

739 Justification: **[TODO]**

740 Guidelines:

- 741 • The answer NA means that the paper poses no such risks.
- 742 • Released models that have a high risk for misuse or dual-use should be released with
 743 necessary safeguards to allow for controlled use of the model, for example by requiring that
 744 users adhere to usage guidelines or restrictions to access the model or implementing safety
 745 filters.
- 746 • Datasets that have been scraped from the Internet could pose safety risks. The authors should
 747 describe how they avoided releasing unsafe images.
- 748 • We recognize that providing effective safeguards is challenging, and many papers do not
 749 require this, but we encourage authors to take this into account and make a best faith effort.

750 12. Licenses for Existing Assets

751 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the
 752 paper, properly credited and are the license and terms of use explicitly mentioned and properly
 753 respected?

754 Answer: **[TODO]**

755 Justification: **[TODO]**

756 Guidelines:

- 757 • The answer NA means that the paper does not use existing assets.
- 758 • The authors should cite the original paper that produced the code package or dataset.
- 759 • The authors should state which version of the asset is used and, if possible, include a URL.
- 760 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 761 • For scraped data from a particular source (e.g., website), the copyright and terms of service
 762 of that source should be provided.
- 763 • If assets are released, the license, copyright information, and terms of use in the package
 764 should be provided. For popular datasets, <https://paperswithcode.com/datasets> has
 765 curated licenses for some datasets. Their licensing guide can help determine the license of
 766 a dataset.
- 767 • For existing datasets that are re-packaged, both the original license and the license of the
 768 derived asset (if it has changed) should be provided.

769 • If this information is not available online, the authors are encouraged to reach out to the
770 asset’s creators.

771 13. New Assets

772 Question: Are new assets introduced in the paper well documented and is the documentation
773 provided alongside the assets?

774 Answer: **[TODO]**

775 Justification: **[TODO]**

776 Guidelines:

- 777 • The answer NA means that the paper does not release new assets.
- 778 • Researchers should communicate the details of the dataset/code/model as part of their
779 submissions via structured templates. This includes details about training, license, limita-
780 tions, etc.
- 781 • The paper should discuss whether and how consent was obtained from people whose asset
782 is used.
- 783 • At submission time, remember to anonymize your assets (if applicable). You can either
784 create an anonymized URL or include an anonymized zip file.

785 14. Crowdsourcing and Research with Human Subjects

786 Question: For crowdsourcing experiments and research with human subjects, does the paper
787 include the full text of instructions given to participants and screenshots, if applicable, as well
788 as details about compensation (if any)?

789 Answer: **[TODO]**

790 Justification: **[TODO]**

791 Guidelines:

- 792 • The answer NA means that the paper does not involve crowdsourcing nor research with
793 human subjects.
- 794 • Including this information in the supplemental material is fine, but if the main contribution
795 of the paper involves human subjects, then as much detail as possible should be included in
796 the main paper.
- 797 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or
798 other labor should be paid at least the minimum wage in the country of the data collector.

799 15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human 800 Subjects

801 Question: Does the paper describe potential risks incurred by study participants, whether such
802 risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals
803 (or an equivalent approval/review based on the requirements of your country or institution) were
804 obtained?

805 Answer: **[TODO]**

806 Justification: **[TODO]**

807 Guidelines:

- 808 • The answer NA means that the paper does not involve crowdsourcing nor research with
809 human subjects.
- 810 • Depending on the country in which research is conducted, IRB approval (or equivalent)
811 may be required for any human subjects research. If you obtained IRB approval, you should
812 clearly state this in the paper.

- 813 • We recognize that the procedures for this may vary significantly between institutions and
814 locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines
815 for their institution.
- 816 • For initial submissions, do not include any information that would break anonymity (if
817 applicable), such as the institution conducting the review.

818 **16. Declaration of LLM Usage**

819 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-
820 standard component of the core methods in this research? Note that if the LLM is used only for
821 writing, editing, or formatting purposes and does *not* impact the core methodology, scientific
822 rigor, or originality of the research, declaration is not required.

823 Answer: **[TODO]**

824 Justification: **[TODO]**

825 Guidelines:

- 826 • The answer NA means that the core method development in this research does not involve
827 LLMs as any important, original, or non-standard components.
- 828 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not be
829 described.